

Kullanılabilirlik: Algı Sınırlarından Türeyen Tasarım Kuralları

ÖFY

September 2026

Özet

Arayüz tasarımının kuralları keyfi bir estetik listesi değil, insan algısının ve belleğinin ölçülebilir sınırlarından türemiş kısıtlardır. Bir düğmeye ulaşma süresi hedefin büyüklüğü ve uzaklığıyla, bir menüden seçim yapma süresi seçenek sayısıyla, bir listenin akılda kalan kısmı kısa süreli belleğin kapasitesiyle belirlenir; bunların hiçbiri tasarımcının tercihinin bağlı değildir. Yazılım mühendisliğindeki bilgi gizleme, modülerlik ve gruplama ilkelerinin arayüz tarafında birebir karşılıkları bulunur: bilgi gizleme kademeli açılıma, modülerlik bileşen sistemine, gruplama Gestalt prensiplerine karşılık gelir. Arayüz tasarımının en sık atlanan kısmı görsel düzen değil, durum kapsamıdır; her ekranın boş, yüklenen, hatalı ve aşırı veri içeren halleri ideal hali kadar tasarım gerektirir. Erişilebilirlik sonradan eklenen bir katman değil, doğru HTML elementinin seçilmesiyle başlayan bir yapı kararıdır. Hızlı hissettirmek ile hızlı olmak farklı problemlerdir; algılanan performans, gerçek sürenin değil belirsizliğin azaltılmasıyla iyileşir. Kullanıcının alışkanlıkları başka ürünlerde olduğu için özgün çözüm üretmenin her zaman bir öğrenme maliyeti vardır. Bu kurallar bütününün karanlık bir kullanımı da mevcuttur: aynı bilgi, kullanıcıyı istemediği bir sonuca yönlendirmek için de kullanılabilir.

Giriş

Ön yüz geliştiricisi (*frontend developer*), sunucudan gelen veriyi kullanıcının gördüğü ve etkileşime girdiği biçime dönüştüren yazılımcıdır. Bu rol iki ayrı bilgi kümesi gerektirir: yazılım mühendisliğinin genel ilkeleri ve arayüz tasarımının kendi kavram dağarcığı. İkinci küme genellikle eksik bırakılır, çünkü tasarımın ayrı bir meslek olduğu varsayılır. Bu varsayım pratikte geçersizdir; tasarımcının verdiği statik ekranlar, uygulama sırasında yüzlerce küçük karar gerektirir ve bu kararları geliştirici verir.

İki alanın kavramları arasında düzenli bir eşleşme vardır. Yazılımda bilgi gizleme (*information hiding*), bir modülün iç ayrıntılarını dış dünyadan saklayıp yerine dar bir arayüz koymaktır; Parnas (1972) tarafından formüle edilmiştir. Arayüz tarafındaki karşılığı kademeli açılımdır (*progressive disclosure*): kullanıcıya her şeyi aynı anda göstermek yerine, ana akışı önde tutup ayrıntıyı talep edene açmak. Modülerlik, yazılımda sistemi bağımsız parçalara bölmektir; arayüzde bileşen (*component*) sistemine ve tasarım sistemine karşılık gelir. Gruplama, yazılımda ilişkili işlevleri aynı modülde toplamaktır; arayüzde Gestalt prensipleriyle, yani ilişkili öğelerin görsel olarak bir arada algılanmasıyla yapılır.

Bu notun kapsamı arayüz tasarımının kavram dağarcığıdır. Belirli bir teknolojinin (React, Angular, Vue) bileşen mimarisi, durum yönetimi ya da derleme süreci kapsam dışındadır. Not, arayüz tasarımını hiç okumamış bir geliştiricinin tek başına okuyabileceği biçimde yazılmıştır; her terim ilk geçtiği yerde tanımlanır ve somut bir örnekle karşılanır.

Notun dayandığı ayırım şudur: arayüz kuralları iki kaynaktan gelir. Bir kısmı ölçülmüş bilişsel sınırlardan (Fitts, Hick, Miller) türer ve deneysel olarak doğrulanmıştır; bir kısmı ise birikmiş uygulama geleneğidir ve bağlama göre değişir. İkisini ayırmak önemlidir, çünkü birinciyi tartışmak anlamsız, ikinciyi tartışmamak ise hatadır.

Görsel Düzen

Hiyerarşi ve boşluk

Görsel hiyerarşi (*visual hierarchy*), ekrandaki öğelerin önem sırasına göre farklı görsel ağırlık taşımasıdır. Ağırlık dört araçla üretilir: boyut, renk kontrastı, yazı kalınlığı ve konum. Hiyerarşinin işlevi, gözün ekrana ilk düştüğü noktayı tasarımcının belirlemesidir.

Hiyerarşinin en yaygın ihlali, bir ekranda eşit ağırlıkta iki birincil eylem bulunmasıdır. Bir formun altında aynı renkte “Kaydet“ ve “İptal“ düğmeleri varsa kullanıcı hangisinin asıl eylem olduğunu ayırt edemez; bu durumda hiyerarşi yok demektir. Çözüm, ikincil eylemi düz metin ya da çerçeveli düğme biçimine indirmektir.

Boş alan (*whitespace, negative space*), doldurulmamış yüzeydir. Doldurulmamış olması onu işlevsiz yapmaz; öğeler arasındaki ayrımı taşıyan asıl araç odur. Boşluk azaldıkça okuma yükü artar, çünkü göz her öğenin sınırını kendisi bulmak zorunda kalır.

Yoğunluk (*density*), birim alana düşen bilgi miktarıdır. Tek bir doğru yoğunluk yoktur; bir yönetim paneli günde saatlerce kullanılacağı ve karşılaştırma gerektirdiği için yüksek yoğunluk ister, bir tanıtım sayfası ise tek bir mesaj taşıdığı için düşük yoğunluk ister. Aynı ürünün farklı ekranları farklı yoğunlukta olabilir.

Hizalama ve ızgara

Hizalama (*alignment*), öğelerin ortak bir eksende toplanmasıdır. Hizalı öğeler düzenli algılanır; hizasızlık göz tarafından hata olarak okunur. Metin sola, sayısal değerler sağa hizalanır. Sayıların sağa hizalanmasının nedeni estetik değil işlevseldir: basamaklar alt alta gelir ve büyüklük karşılaştırması gözle yapılabilir hale gelir.

Izgara sistemi (*grid system*), sayfanın görünmez sütunlara bölünmesidir. On iki sütunun yaygınlaşmasının nedeni bölünebilirliğidir; ikiye, üçe, dörde ve altıya tam bölünür, dolayısıyla çoğu yerleşim ihtiyacını karşılar. Izgaranın asıl faydası tek bir ekranda değil, farklı ekranların aynı ritmi taşımasındadır.

Optik hizalama (*optical alignment*), matematiksel merkezin görsel merkezle örtüşmediği durumlarda yapılan düzeltmedir. Bir oynat ikonu (üçgen) bir dairenin tam matematiksel merkezine konduğunda sola kaymış görünür, çünkü üçgenin görsel ağırlık merkezi geometrik merkezinden farklıdır. Bu durumda ikon birkaç piksel sağa kaydırılır. Ölçüt gözdür, sayı değil.

Gestalt prensipleri

Gestalt psikolojisi, algının parçaların toplamından farklı bir bütün ürettiğini öne süren ekoldür; temel prensipleri Wertheimer (1923) tarafından formüle edilmiştir. Arayüz tasarımının gruplama araçları doğrudan bu prensiplerden gelir.

Yakınlık

Yakınlık (*proximity*), birbirine yakın duran öğelerin ilişkili algılanmasıdır. Gestalt prensipleri içinde en güçlüsüdür; çerçeve ya da çizgi kullanmadan grup oluşturabilir.

Kuralın uygulanabilir hali şudur: grup içi boşluk, gruplar arası boşluktan küçük olmalıdır. Bir formda ad, soyad ve e-posta alanları arasında sekiz piksel, “Kişisel Bilgiler” bloğu ile “Adres” bloğu arasında otuz iki piksel boşluk bırakıldığında gruplar kendiliğinden okunur. Bu oran bozulduğunda, yani tüm boşluklar eşitlendiğinde, kullanıcı hangi alanın hangi başlığa ait olduğunu ancak okuyarak çıkarabilir.

Aynı prensip etiket yerleşimini de belirler. Bir etiket, ait olduğu alandan çok bir sonraki alana yakınsa, kullanıcı onu yanlış alana bağlar.

Benzerlik ve ortak bölge

Benzerlik (*similarity*), benzer görünen öğelerin aynı işlevi taşıdığının varsayılmasıdır. Bir üründe tüm yıkıcı eylemler kırmızıysa, kullanıcı kırmızı bir düğme gördüğünde onun bir şey sileceğini varsayar. Bu varsayım tasarımcı istemese de kurulur; dolayısıyla tutarsızlık doğrudan hataya dönüşür.

Prensibin tersi de geçerlidir: farklı işler farklı görünmelidir. Aynı görünen iki düğmeden birinin kaydetmesi diğerinin göndermesi, kullanıcının ayrımı ezberlemesini gerektirir.

Ortak bölge (*common region*), ortak bir alan içine alınan öğelerin gruplanmasıdır. Kart, çerçeve ve arka plan bloğu bu prensibin araçlarıdır. Yakınlık yeterli olmadığında, yani gruplar arasında yeterli boşluk bırakılamadığında devreye girer.

Kapanış, süreklilik, şekil-zemin, ortak kader

Kapanış (*closure*), beynin eksik şekli tamamlamasıdır. Kesik çizgili bir dikdörtgen yine dikdörtgen olarak okunur. Yükleme sırasında kullanılan iskelet ekranların (*skeleton screen*) işe yaramasının nedeni budur: içerik henüz yokken bile yapı tanınır.

Süreklilik (*continuity*), gözün bir çizgiyi ya da yönü takip etmesidir. Alt alta dizilmiş kartlar bir sütun oluşturur ve göz aşağı doğru akar. Bu akışı bozan bir öğe, istense de istenmese de dikkat çeker.

Şekil-zemin (*figure-ground*), neyin ön planda neyin arka planda olduğunun ayırt edilmesidir. Bir kip penceresi (*modal*) açıldığında arkasının karartılmasının nedeni bu ayrımı zorlamaktır. Kontrast yetersizse kullanıcı hangi katmanın etkin olduğunu anlayamaz ve arkadaki öğelere tıklamayı dener.

Ortak kader (*common fate*), birlikte hareket eden öğelerin ilişkili algılanmasıdır. Bir akordiyon açılırken altındaki içeriğin birlikte aşağı kayması, o içeriğin o gruba ait olduğunu söyler. Hareket burada süs değil, yapı bilgisidir.

Tipografi ve Renk

Tipografi

Ölçek (*type scale*), yazı boyutlarının rastgele değil bir orana göre belirlenmesidir. Bir taban boyut seçilir ve sabit bir çarpanla (yaygın olarak 1,25 ya da 1,333) çoğaltılır; 14, 16, 20, 25, 31 gibi bir dizi elde edilir. Faydası, her yeni ihtiyaçta yeni bir sayı uydurmayı engellemesidir.

Ölçü (*measure*), bir satırdaki karakter sayısıdır. Bringhurst (1992), gövde metni için 45–75 karakterlik aralığı standart olarak verir. Satır bundan uzun olduğunda göz satır sonundan bir sonrakinin başına dönerken hedefi kaybeder; kısa olduğunda okuma çok sık bölünür. Bu, arayüzde metin sütunlarına azami genişlik konmasının nedenidir; ekran geniş diye metin sütununu da genişletmek okunabilirliği düşürür.

Satır aralığı (*line height, leading*), satırlar arasındaki dikey boşluktur. Gövde metni için yazı boyutunun yaklaşık 1,5 katı işlevsel bir başlangıçtır. Başlıklarda bu oran düşer (1,1–1,2), çünkü büyük punto zaten kendi boşluğunu taşır; başlıkta gövde oranı kullanıldığında satırlar birbirinden kopar.

Harf aralığı iki ayrı kavramla ifade edilir. Tracking, bir metin bloğunun tümüne uygulanan aralık ayarıdır; kerning ise belirli iki harf arasındaki tekil düzeltmedir. Büyük başlıklarda tracking hafifçe kısılır, tamamı büyük harfle yazılan küçük etiketlerde açılır.

Font eşleştirmede (*font pairing*) tek gerçek hata, birbirine çok benzeyen iki fontun bir arada kullanılmasıdır; bu, fark edilmeyen ama rahatsız eden bir tutarsızlık üretir. İki font ya açıkça farklı olmalı ya da tek fontun farklı ağırlıkları kullanılmalıdır.

Hiyerarşi yalnızca boyutla kurulmaz. Aynı boyutta ama daha kalın bir metin, daha büyük ama ince bir metinden görsel olarak daha güçlü durabilir. Bu, sınırlı dikey alanda hiyerarşi kurmanın pratik yoludur.

Renk

Kontrast oranı (*contrast ratio*), metin ile arka plan arasındaki parlaklık farkının sayısal ölçüsüdür. WCAG 2.1 (W3C, 2018), AA düzeyi için normal metinde en az 4,5:1, büyük metinde 3:1 oranını şart koşar. Açık gri zemin üzerine beyaz metin, tasarımda sık karşılaşılan ve bu eşiği geçemeyen bir kalıptır.

Tonal palet (*tonal palette*), tek bir renkten açıktan koyuya doğru dokuz-on tonluk bir dizi üretilmesidir; genellikle 50, 100, 200 ... 900 biçiminde numaralanır. Arka plan için en açık tonlar, kenarlık için ara tonlar, metin için koyu tonlar kullanılır. Böyle bir dizi üretilmediğinde, her yeni ihtiyaçta ara bir renk uydurulur ve palet zamanla dağılır.

Anlamsal renk (*semantic color*), rengin belirli bir anlam taşımasıdır: kırmızı hata, yeşil başarı, sarı uyarı, mavi bilgi. İki kural geçerlidir. Bu eşleşme ürün içinde tutarlı kalmalıdır; ve renk, anlamın tek taşıyıcısı olmamalıdır. Kırmızı-yeşil renk körlüğü toplumda yaygın olduğundan, yalnızca renkle ayrılan başarı

ve hata durumları bir kısım kullanıcı için ayırt edilemez. Renge ikon ya da metin eşlik etmelidir.

Vurgu rengi (*accent color*), dikkat çekmesi gereken tek renktir: birincil eylem, etkin sekme, seçili durum. Az kullanıldığı için işlev görür; ekrana yayıldığında vurgu olmaktan çıkar. 60-30-10 kuralı bu dengeyi kabaca formüle eder: yüzeyin yaklaşık %60'ı baskın nötr renk, %30'u ikincil renk, %10'u vurgudur.

Etkileşim ve Kullanılabilirlik Yasaları

Etkileşimin temel kavramları

Sağlayan (*affordance*), bir nesnenin kendi biçimiyle ne yapılabileceğini bildirmesidir; kavram Gibson'dan alınarak Norman (1988) tarafından tasarım alanına taşınmıştır. Kabarık bir yüzey basılabilir, girintili bir kutu içine yazı yazılabilir görünür.

Gösteren (*signifier*), sağlayanı işaret eden algılanabilir ipucudur. Norman ayrımı şöyle kurar: kapının itilebilir olması sağlayandır, üstündeki "İTİNİZ" yazısı gösterendir. Arayüzde imlecin el biçimine dönmesi, bağlantının altının çizili olması ve düğmenin gölgesi gösterenlerdir. Tamamen düz tasarımın yarattığı sorun, sağlayanları koruyup gösterenleri kaldırmasıdır; nesne tıklanabilir ama tıklanabilir görünmez.

Geri bildirim (*feedback*), her eylemin bir karşılık üretmesidir. Tıklanan düğme görsel olarak tepki vermeli, gönderilen form bir sonuç bildirmelidir. Sesizlik kullanıcıya işlemin başarısız olduğunu düşündürür ve ikinci kez tıklamaya yöneltir; bu da çift kayıt gibi somut hatalar üretir.

Sistem durumunun görünürlüğü (*visibility of system status*), Nielsen'in (1994) on sezgisel kuralının ilkidir. Sistem her an ne yaptığını bildirmelidir: yükleniyor, kaydedildi, bağlantı koptu, üç öğeden ikisi işlendi. Belirsizlik, yavaşlıktan daha rahatsız edicidir; kullanıcı beklemeye değil, ne beklediğini bilmemeye tahammülsüzdür.

Doğrudan manipülasyon (*direct manipulation*), nesnenin ayrı bir arayüz üzerinden değil, kendisi üzerinden değiştirilmesidir. Sürükle-bırakla sıralama, köşesinden tutup boyutlandırma, yerinde düzenleme bu kapsamdadır. Ayrı bir form açıp orada düzenlemeye göre daha az adım gerektirir ve sonucu anında gösterir.

Mikro etkileşim (*micro-interaction*), tek amaçlı, kısa etkileşim anıdır: beğeni düğmesinin animasyonu, anahtarın kayması, alanın odaklandığında kenarlığının renklenmesi. Ürünün algılanan kalitesini büyük ölçüde bunlar belirler.

İyimser arayüz (*optimistic UI*), sunucudan yanıt beklemeden arayüzün güncellenmesidir. Beğeni düğmesine basıldığında simge hemen dolar, istek arka planda gider, hata dönerse geri alınır. Hızlı hissettirir ancak geri alma senaryosunun açıkça tasarlanmasını gerektirir. Para transferi ve silme gibi geri alınamayan işlemlerde kullanılmaz; kötümser yaklaşım, yani sonucu bekleyip göstermek, bu tür işlemlerde doğrudur.

Geri alma, onaya tercih edilir. "Emin misiniz?" biçimindeki onay penceresi kısa sürede okunmadan kapatılan bir refleks haline gelir ve koruma işlevini yitirir; işlemi yapıp geri alma imkânı sunmak ise gerçekten kullanılır. Onay penceresi, geri alınması teknik olarak mümkün olmayan işlemlere saklanmalıdır.

Ölçülmüş yasalar

Bu başlık altındakiler deneysel psikolojiden gelir ve tasarım tercihinin bağlıdır.

Fitts yasası

Fitts (1954), bir hedefe ulaşma süresinin hedefin uzaklığıyla artıp büyüklüğüyle azaldığını göstermiştir. İlişki logaritmiktir; hedefi iki katına çıkarmak süreyi yarıya indirmez ama belirgin biçimde kısaltır.

Uygulamadaki üç sonucu şunlardır. Sık kullanılan eylemler büyük ve yakın olmalıdır. Dokunmatik ekranda dokunma hedefi en az 44×44 piksel olmalıdır; görsel boyut daha küçük olabilir, ancak tıklanabilir alan büyütülmelidir. Ekranın kenarları ve köşeleri pratikte sonsuz büyüklükte hedef sayılır, çünkü imleç oraya çarpıp durur; bu nedenle masaüstü işletim sistemlerinde menü çubuğu ekranın en üstüne yerleştirilir.

Hick yasası

Hick (1952) ve Hyman (1953), karar süresinin seçenek sayısı ile logaritmik olarak arttığını göstermiştir. On beş maddelik düz bir menü, gruplara ayrılmış on beş maddeden daha yavaş taranır.

Uygulamadaki karşılıkları: seçenekleri gruplamak, varsayılan bir seçenek belirlemek, az kullanılan seçenekleri kademeli açılımın arkasına almak. Yasanın sınırı şudur: seçenek sayısını azaltmak, işi yapılamaz hale getirecek kadar ileri götürülmemelidir. Tesler yasası bu sınırı açıklar.

Miller yasası

Miller (1956), kısa süreli belleğin yaklaşık yedi (artı eksi iki) birim tutabildiğini bildirmiştir. Sonraki araştırmalar bu sayıyı daha düşük tahmin etse de, sınırın varlığı tartışmalı değildir.

Pratikteki karşılığı parçalamadır (*chunking*): uzun bir sayıyı gruplara bölmek, uzun bir formu adımlara ayırmak, uzun bir metni paragraflara bölmek. Telefon numarasının üçerli ve ikişerli gruplar halinde yazılması bu ilkenin en eski uygulamasıdır.

Tesler yasası

Tesler yasası, her sistemin azaltılamaz bir karmaşıklığı olduğunu söyler. Bu karmaşıklık yok edilemez, yalnızca taraflar arasında paylaştırılabilir: ya kullanıcı taşır ya arayüz.

Örnek: bir tarih girişi. Kullanıcıya hangi formatta gireceğini sormak karmaşıklığı ona yıkmaktır. Girilen değeri esnek biçimde ayrıştırıp normalleştirmek ise karmaşıklığı arayüzün üstlenmesidir. Aynı iş yapılır, yük yer değiştirir.

Doherty eşiği

Doherty ve Thadani (1982), sistem yanıt süresi 400 milisaniyenin altına indiğinde kullanıcı üretkenliğinin belirgin biçimde arttığını ölçmüştür. Bu eşiğin üzerinde dikkat dağınık ve kullanıcı başka işe yönelir.

Gerçek işlem süresi kısaltılamadığında dahi algılanan süre kısaltılabilir: iskelet ekran, iyimser güncelleme ve önceden yükleme bu amaca hizmet eder.

Bellek ve dikkat etkileri

Von Restorff etkisi (1933), bir dizide farklı olan öğenin daha iyi hatırlandığını söyler. Beş nötr düğme arasındaki tek renkli düğme göze çarpar. Bu, vurgunun neden az kullanılması gerektiğinin bilişsel gerekçesidir.

Zeigarnik etkisi (1927), yarım kalan işlerin tamamlananlardan daha iyi hatırlandığını söyler. İlerleme çubuğu, “profiliniz %60 tamamlandı“ bildirimi ve tamamlanmamış kontrol listesi bu etkiyi kullanır.

Sıra konumu etkisi (*serial position effect*), bir listenin başındaki ve sonundaki öğelerin ortadakilerden daha iyi hatırlandığını söyler. En önemli iki menü maddesi listenin iki ucuna yerleştirilir.

Zirve-son kuralı (*peak-end rule*), Kahneman ve arkadaşlarının (1993) çalışmalarına dayanır: bir deneyim, en yoğun anı ve bitişi üzerinden hatırlanır, ortalaması üzerinden değil. Bir ödeme akışının sonundaki teşekkür ekranı, tüm sürecin hatırlanma kalitesini orantısız biçimde belirler.

Jakob yasası

Nielsen (2000) tarafından formüle edilen bu kural, kullanıcının zamanının büyük kısmını başka ürünlerde geçirdiğini ve dolayısıyla bir ürünün de onlar gibi çalışmasını beklediğini söyler. Logonun sol üstte olması ve ana sayfaya bağlanması, sepetin sağ üstte bulunması bu beklentilerdendir.

Kuralın anlamı özgün çözümün yasak olduğu değil, bir maliyeti olduğudur. Yerleşik kalıptan sapmak, kullanıcıya öğrenme yükü bindirir; bu yükün karşılığında somut bir kazanç sağlanmalıdır.

Estetik-kullanılabilirlik etkisi

Kurokawa ve Kashimura (1995), kullanıcıların estetik olarak beğendikleri arayüzleri daha kullanışlı bulduğunu göstermiştir. İki sonucu vardır. Olumlusu, görsel özenin kullanıcıya tolerans kazandırmasıdır. Olumsuz, güzel bir arayüzün kullanılabilirlik sorunlarını gizleyebilmesi, dolayısıyla estetik memnuniyetin kullanılabilirlik testinin yerine geçememesidir.

Bilgi Mimarisi ve İçerik Tasarımı

Bilginin düzenlenmesi

Bilgi mimarisi (*information architecture*), içeriğin nasıl organize edildiğine dair karardır: hangi içerik hangi başlığın altında bulunur, kategoriler nasıl adlandırılır, hiyerarşi kaç kademedir. Bu karar arayüz tasarımından önce gelir; yanlış kurulmuş bir bilgi mimarisi, hiçbir görsel düzenlemeyle telafi edilemez.

Kart ayırma (*card sorting*), bilgi mimarisini kullanıcının zihinsel modelinden kurmak için kullanılan yöntemdir: içerik başlıkları kullanıcılara verilir ve kendi kategorilerine ayırmaları istenir. Tasarımcının kendi kafasındaki kategorilerin kullanıcınıninkiyle örtüşmemesi olağan durumdur.

Kademeli açılım (*progressive disclosure*), bilginin katmanlar halinde sunulmasıdır. Ödeme formunda kartın temel alanları önde, fatura adresi bir onay kutusunun arkasındadır. Ayarlar sayfasında temel ayarlar üstte, seyrek kullanılanlar bir akordiyon içindedir. Hata bildiriminde kısa mesaj görünür, teknik ayrıntı bir bağlantının arkasındadır.

Bilgi kokusu (*information scent*), kullanıcının doğru yolda olduğunu hissettiren ipuçlarıdır; kavram Pirolli ve Card'ın (1999) bilgi arama kuramından gelir. Bağlantı metni gideceği yeri bildirmelidir. “Buraya tıklayın“ kokusuz, “Fatura geçmişi“ kokulu bir bağlantıdır. Koku zayıf olduğunda kullanıcı deneme yanılmaya başlar ve her yanlış deneme geri dönüş maliyeti üretir.

Okuma davranışı

Taranabilirlik (*scannability*), metnin baştan sona okunmadan gözle taranabilmesidir. Krug (2000) kullanıcıların sayfaları okumadığını, taradığını ve ilk makul seçeneği seçtiğini bildirir. Bu davranışa uygun metin kısa paragraflar, alt başlıklar, kalınlaştırılmış anahtar kelimeler ve madde listeleri içerir.

F ve Z kalıpları, gözün sayfada izlediği yolu betimler. Yoğun metinli sayfalarda göz F biçiminde hareket eder: üstte yatay bir tarama, ardından kısalan ikinci bir yatay tarama, sonra sol kenarda dikey iniş. Görsel ağırlıklı ve az içerikli sayfalarda Z biçimi görülür. Önemli öğeler bu yolların üzerine yerleştirilir.

Katlanma çizgisinin üstü (*above the fold*), sayfa açıldığında kaydırmadan görünen alandır. Terim gazete katlama pratiğinden gelir. Kullanıcının kalma kararını büyük ölçüde burada verdiği kabul edilir; dolayısıyla ürünün ne olduğu ve kullanıcının ne yapması gerektiği bu alanda bildirilmelidir.

İçerik önceliğinde tasarım (*content-first design*), tasarımın gerçek içerikle yapılmasıdır. Yer tutucu metinle (*lorem ipsum*) çalışıldığında her düzen kusursuz görünür; uzun başlık, üç karakterlik isim, boş alan ve taşan tablo yalnızca gerçek içerikte ortaya çıkar.

Arayüz Durumları ve Performans Algısı

Durum kapsamı

Her ekranın ideal halinin dışında en az dört hali vardır ve bunların tasarlanmaması ön yüz geliştirmesinin en yaygın eksiğidir.

Boş durum (*empty state*), veri bulunmadığında görünen haldir. Boş bir tablo yeterli değildir; ekran ne olduğunu, neden boş olduğunu ve ilk adımın ne olduğunu bildirmelidir. Boş durum aynı zamanda öğretim alanıdır, çünkü yeni kullanıcının ürünle ilk karşılaştığı yerdir.

Yükleme durumu (*loading state*), veri beklenirken görünen haldir. Döner simge (*spinner*) en basit ama en az bilgi veren biçimdir; yalnızca bir şeyin sürdüğünü söyler, ne kadar süreceğini söylemez. Süre ölçülebiliyorsa ilerleme çubuğu, yapı biliniyorsa iskelet ekran tercih edilir.

İskelet ekran (*skeleton screen*), gelecek içeriğin gri bloklarla taklididir. Döner simgeden daha hızlı hissettirmesinin nedeni kapanış prensibidir: yapı görünür olduğu için göz yerleşimi önceden kavrar ve içerik geldiğinde yeniden konumlanmaz.

Hata durumu (*error state*) üç bilgiyi taşımalıdır: ne olduğu, mümkünse neden olduğu ve kullanıcının şimdi ne yapabileceği. “Bir hata oluştu” bu üçünden hiçbirini karşılamaz.

Sınır durumları (*edge cases*), ideal olmayan veri koşullarıdır: çok uzun isim, üç bin satırlık liste, tek sonuç veren arama, yetkisiz kullanıcı, çevrimdışı bağlantı. Bunlar istisna değil, üretimde düzenli olarak karşılaşılan hallerdir.

Algılanan performans

Algılanan performans (*perceived performance*), kullanıcının hissettiği süredir ve ölçülen süreyle aynı değildir. Aynı üç saniye, boş bir ekranda uzun, iskelet ekranla kısa hissedilir. Optimizasyonun bir kısmı gerçek süreyi, bir kısmı ise belirsizliği hedefler.

Düzen kayması (*layout shift*), içerik yüklendikçe sayfadaki öğelerin yer değiştirmesidir. En yaygın nedeni, görsellere ve gelecek içeriğe önceden boyut verilmemesidir. Sonucu yalnızca rahatsızlık değil, somut hatadır: kullanıcı bir düğmeye tıklamak üzereyken düzen kayar ve başka bir şeye tıklar.

Tembel yükleme (*lazy loading*), görünür alana girmemiş içeriğin sonraya bırakılmasıdır. İlk yükleme süresini kısaltır ancak yer tutucu bırakılmadan uygulandığında düzen kayması üretir; iki teknik birlikte düşünülmelidir.

Önceden getirme (*prefetch, preload*), kullanıcının gitme ihtimali yüksek olan içeriğin önceden yüklenmesidir. Bir bağlantının üzerine gelindiğinde hedef sayfanın hazırlanması tipik örnektir.

Geciktirme (*debounce*) ve kısıtlama (*throttle*), sık tetiklenen olayların seyreltilmesidir. Arama kutusunda her tuş vuruşunda istek göndermemek teknik bir optimizasyon gibi görünse de asıl etkisi deneyimdedir: sonuç listesinin her harfte yeniden çizilmesi okumayı imkânsız kılar.

Güncelliğın bildirilmesi (*stale data indication*), verinin ne zaman güncellendiğinin gösterilmesi ve yenileme imkânı sunulmasıdır. Sayısal gösterge panellerinde bu bilgi olmadan, kullanıcı eski veriye dayanarak karar verebilir.

Bileşen Kalıpları ve Veri Gösterimi

Katman ve bildirim bileşenleri

Kip pencere (*modal*), sayfanın üzerine açılan ve arkasını etkisizleştiren katmandır; akışı keser ve odak gerektirir. Çekmece (*drawer*) kenardan açılır ve arkadaki bağlamı görünür bırakır. Açılır balon (*popover*) küçük ve geçicidir. Yanlış olanın seçilmesi en sık görülen bileşen hatasıdır: bağlamın korunması gereken bir işlem kip pencereye konduğunda kullanıcı arkadaki veriyi göremez ve pencereyi kapatıp açmak zorunda kalır.

Geçici bildirim (*toast, snackbar*), kısa süre görünüp kendiliğinden kaybolan bildirimdir. Kendiliğinden kaybolduğu için kritik bilgi taşıyamaz; okunmadan kaybolan bir hata mesajı hiç gösterilmemiş sayılır. Kalıcı bildirim için sayfa genelini ilgilendiren durumlarda üst şerit (*banner*), tek bir alanı ilgilendiren durumlarda alanın yanında satır içi uyarı kullanılır.

İpucu balonu (*tooltip*) yalnızca tamamlayıcı bilgi taşır. İçinde tıklanabilir öğe bulunmamalıdır ve dokunmatik cihazlarda üzerine gelme (*hover*) durumu olmadığından tek bilgi kaynağı yapılmamalıdır.

Yapı ve seçim bileşenleri

Sekme (*tabs*) paralel ve eşdeğer içerikler için, akordiyon uzun içeriği katlamak için, adım göstergesi (*stepper*) sıralı ve zorunlu akış için kullanılır. Üçü birbirinin yerine kullanıldığında kullanıcı içeriğin yapısı hakkında yanlış bilgilendirilir.

Seçim kontrollerinde ayırım şöyledir: iki ya da üç seçenek için radyo düğmesi veya bölümlü kontrol (*segmented control*), daha fazlası için açılır liste. Açılır liste seçenekleri gizler, radyo düğmesi hepsini görünür tutar; seçeneklerin karşılaştırılması gerekiyorsa görünür olmaları gerekir.

Anahtar (*toggle*) ile onay kutusu (*checkbox*) arasındaki ayırım işlevseldir. Anahtar anında etki eder, onay kutusu bir kaydetme işlemi bekler. Bu ayırım karıştırıldığında kullanıcı değişikliğin kaydedilip kaydedilmediğini bilemez.

Veri gösterimi

Tablo, liste ve kart ızgarası farklı işler için uygundur: karşılaştırma yapılacaksa tablo, tarama yapılacaksa liste, görsel ağırlıklı içerik varsa kart ızgarası.

Toplu eylemler (*bulk actions*) çoklu seçim, seçili sayısının gösterimi, tümünü seçme ve görünen sayfanın dışını da kapsayan seçim gibi alt problemleri beraberinde getirir. Genellikle sonradan eklenmeye çalışılır ve mevcut düzeni bozar; başından planlanması gerekir.

Sayfalama (*pagination*), sonsuz kaydırma (*infinite scroll*) ve “daha fazla yükle” düğmesi arasındaki seçim, içeriğin niteliğine bağlıdır. Sayfalama konum bilgisi verir ve geri dönülebilir kılar. Sonsuz kaydırma keşif akışlarına uygundur ancak sayfa altındaki içeriğe erişimi pratikte imkânsızlaştırır. “Daha fazla yükle” ikisinin arasında durur ve kullanıcıya kontrol bırakır.

Kesme (*truncation*), sığmayan metnin kısaltılmasıdır. Kesilen metnin tamamına bir yolla erişilebilmelidir. Dosya adlarında ortadan kesme tercih edilir, çünkü uzantı bilgi taşır; başlıklarda sondan kesme uygundur.

Biçimlendirme kararları da tasarım kararıdır. Sayılar sağa hizalanır ve ondalık basamak sayısı sabit tutulur; para birimi ve büyüklük birimi açıkça yazılır.

Arama ve filtreleme

Arama, filtre ve sıralama farklı işler yapar: arama bulmaya, filtre daraltmaya, sıralama önceliklendirmeye yarar. Aynı bölgede bulunmaları üçün aynı şey yapmaz.

Çok yönlü arama (*faceted search*), kategori, fiyat ve marka gibi birden fazla filtre boyutunun bir arada kullanılmasıdır. İki kural geçerlidir: her filtre seçeneğinin yanında kaç sonuç üreteceği gösterilmeli ve sıfır sonuç üretecek seçenekler etkisizleştirilmelidir. Aksi halde kullanıcı boş sonuç üreten kombinasyonları deneyerek zaman kaybeder.

Filtrelerin kalıcılığı, bir öğenin detayına girip geri dönüldüğünde filtrelerin korunmasıdır. Filtre durumu adres satırında (*URL*) tutulduğunda bu sorun kendiliğinden çözülür ve ek olarak paylaşılabirlik kazanılır.

Sonuç bulunamadığında gösterilen ekran üç şey yapmalıdır: ne arandığını tekrarlamak, yazım önerisi sunmak ve filtreleri temizlemek için bir yol vermek.

Otomatik tamamlama (*typeahead, autocomplete*) gecikmeli tetiklenmeli, klavyeyle gezilebilmeli ve eşleşen bölüm görsel olarak vurgulanmalıdır.

Form Tasarımı

Form, kullanıcıdan veri alınan ve dolayısıyla hata olasılığının en yüksek olduğu arayüz parçasıdır. Wroblewski (2008) form tasarımının temel kalıplarını sistematik biçimde derlemiştir.

Etiket yerleşiminde (*label placement*) etiketin alanın üstünde bulunması çoğu durumda en iyi sonucu verir: tek sütunlu tarama sağlar, uzun etiketlerde bozulmaz ve dar ekranlarda çalışır. Yer tutucu metnin (*placeholder*) etiket yerine kullanılması yaygın bir hatadır; kullanıcı yazmaya başladığında kaybolur ve alanın ne istediği unutulur. Yer tutucu örnek gösterir, yardımcı metin ise kalıcı kural bildirir; kural niteliğindeki bilgi yardımcı metne yazılır.

Satır içi doğrulama (*inline validation*), hatanın form gönderilmeden bildirilmesidir. Zamanlaması önemlidir: kullanıcı yazarken doğrulamak erken ve rahatsız edicidir, çünkü henüz tamamlanmamış girdi hatalı görünür; alandan çıkıldığında doğrulamak uygun zamandır.

Girdi maskesi (*input masking*), yazılırken biçimin otomatik uygulanmasıdır: kart numarasının dörderli ayrılması, tarih alanına ayraç eklenmesi. Hata oranını düşürür ve girilen değer doğruluğunu gözle denetlenebilir kılar.

Akıllı varsayılanlar (*smart defaults*), en olası değer önceden seçili gelmesidir: ülke, para birimi, bugünün tarihi. Kullanıcıların çoğu varsayılanı dokunmaz; dolayısıyla varsayılanın doğruluğu, alanın kendisinden daha etkilidir.

Bağışlayıcı biçim (*forgiving format*), kullanıcının girdiğini reddetmek yerine normalleştirmektir. Boşluklu yazılmış kart numarası, farklı ayraçla yazılmış tarih ve başında sıfır bulunan telefon numarası kabul edilip düzeltilmelidir. Bu, Tesler yasasının form tarafındaki uygulamasıdır: karmaşıklığı arayüz üstlenir.

Hatadan kurtulma (*error recovery*), hata sonrasında kullanıcının devam edebilmesidir. Girilen veri silinmemeli, odak hatalı alana taşınmalı ve mesaj ne yapılacağını söylemelidir.

Veri en aza indirme (*data minimization*), her sorulan alanın bir gerekçesinin bulunmasıdır. Gereksiz alan yalnızca gizlilik sorunu değil, doğrudan dönüşüm kaybıdır.

Navigasyon

Yön bulma (*wayfinding*), kullanıcının her an üç soruyu yanıtlayabilmesidir: neredeyim, nereye gidebilirim, nasıl geri dönerim. Etkin menü vurgusu, sayfa başlığı ve kırıntı yolu bu soruların araçlarıdır.

Kırıntı yolu (*breadcrumb*), hiyerarşideki konumu gösteren yoldur. Derin yapılarda işlevseldir; iki kademeli bir sitede yer kaplamaktan başka iş görmez.

Menü derinliği (*nav depth*), hiyerarşinin kaç kademe olduğudur. Derinlik arttıkça kullanıcının hedefi bulma olasılığı düşer; geniş ve sık yapı genellikle dar ve derin yapıdan daha iyi sonuç verir.

Geri düğmesi davranışı, tarayıcı geri tuşunun beklenen sonucu üretmesidir. Bir kip pencere açıksa geri tuşu onu kapatmalı, kullanıcıyı siteden çıkarmamalıdır. Bu davranış çoğu tek sayfa uygulamasında ayrıca yazılmadığı için bozuktur.

Derin bağlantı (*deep linking*), her uygulama durumunun bir adrese karşılık gelmesidir: filtrelenmiş liste, açık sekme, seçili öğe. Bunun üç faydası vardır: durum paylaşılabilir, sayfa yenilendiğinde kaybolmaz ve tarayıcı geçmişini anlamlı çalışır.

Yapışkan öğeler (*sticky elements*), kaydırma sırasında sabit kalan başlık, eylem çubuğu ya da tablo sütunudur. Dikey alanı sürekli işgal ettikleri için özellikle dar ekranlarda ölçülü kullanılmalıdır.

Kaydırma davranışının iki ayrıntısı sıkça atlanır. Kaydırma konumunun geri dönüşte korunması (*scroll restoration*), listeden detaya girip dönen kullanıcının yerini kaybetmemesini sağlar. Kaydırma çıpalama (*scroll anchoring*), üstte yeni içerik yüklendiğinde sayfanın zıplamasını engeller.

İç içe kaydırma alanları kullanıcı için belirsizlik üretir; hangi alanın kaydığı anlaşılmaz. Mümkün olduğunca tek kaydırma eksenini bırakılmalıdır.

Katman sırası (*z-index*), hangi öğenin hangisinin üstünde çizileceğini belirler. Rastgele sayılar yerine adlandırılmış katmanlar tanımlanır: açılır liste, kip pencere, geçici bildirim. Aksi halde katman çakışmaları rastgele düzeltme sayılarıyla çözülmeye çalışılır.

Responsive Tasarım ve Platform

Ekran genişliğine uyum

Önce mobil (*mobile-first*), tasarıma küçük ekrandan başlayıp genişletmektir. Faydası teknik değil, zihinseldir: küçük ekran neyin zorunlu olduğuna karar vermeye zorlar. Masaüstünden başlandığında, daralma sırasında neyin çıkarılacağı belirsiz kalır.

Kırılma noktaları (*breakpoints*), düzenin değiştiği ekran genişlikleridir. Cihaz isimlerine göre değil, içeriğin bozulduğu noktaya göre seçilmelidir; cihaz boyutları sürekli değişirken içeriğin sınırları sabittir.

Akışkan tipografi (*fluid typography*), yazı boyutunun ekran genişliğiyle sürekli değişmesidir. `clamp()` işleviyle alt sınır, tercih edilen ölçek ve üst sınır verilir; böylece kırılma noktalarında ani sıçrama olmaz.

Konteyner sorguları (*container queries*), bileşenin ekran genişliğine değil, içinde bulunduğu kabın genişliğine göre düzen değiştirmesidir. Aynı kart bileşeni geniş bir alanda yatay, dar bir sütunda dikey düzene geçebilir. Bu, bileşen bağımsızlığını gerçek anlamda mümkün kılan mekanizmadır; ekran sorgularıyla bileşen her zaman bulunduğu yerden habersizdir.

Başparmak bölgesi (*thumb zone*), telefonu tek elle tutan başparmağın rahat ulaştığı alandır ve ekranın alt orta kısmına karşılık gelir. Birincil eylemler bu bölgeye, yıkıcı eylemler ise ulaşılması güç üst köşelere yerleştirilir.

Platform ve ekran koşulları

Platform gelenekleri, işletim sistemlerinin kendi kalıplarıdır: geri hareketi, paylaşım ikonu, sekme çubuğunun konumu, tarih seçici biçimi. Yerel uygulamada platformun kalıbına uyulur; web uygulamasında her iki platforma birden benzemeye çalışmak yerine web kalıbında kalmak genellikle daha tutarlı sonuç verir.

CSS pikseli ile fiziksel piksel aynı birim değildir; aralarındaki oran cihaz piksel oranıdır (*device pixel ratio*). Bu nedenle bit eşlem görsellerin iki ve üç katı çözünürlükte sürümleri sağlanır, ikonlar ise vektör biçiminde tutulur.

Güvenli alan (*safe area*), çentik ve alt sistem çubuğu gibi ekranın işletim sistemi tarafından kullanılan bölgeleridir. Sabit konumlu öğeler bu alanlara girdiğinde tıklanamaz hale gelir.

Yakınlaştırma ve metin ölçeği, kullanıcının tarayıcı yazısını büyütmesi ya da sistemde metin boyutunu artırmış olmasıdır. Sabit piksel yükseklikli kutular bu durumda içeriği taşırır; yükseklik içeriğe göre belirlenmelidir.

Yazdırma biçimlendirmesi (*print stylesheet*), fatura, rapor ve bilet gibi ekranlar için gereklidir: navigasyon gizlenir, renkler sadeleştirilir ve sayfa kırılmaları ayarlanır.

Çevrimdışı durum (*offline state*), bağlantı bulunmadığında arayüzün davranışdır: salt okunur moda geçme, işlemleri kuyruğa alma ve durumu bildirme.

Çakışma çözümü (*conflict resolution*), iki kullanıcının aynı kaydı eş zamanlı düzenlemesi durumunda ne olacağına dair karardır. Son yazanın kazanması,

uyarı gösterilmesi ya da deęişikliklerin birleřtirilmesi seenekleri vardır; bu bir veri tabanı kararı gibi görünse de sonucu kullanıcı deneyimidir.

Saat dilimi gösterimi, hangi saatin gösterildięinin açıka bildirilmesini gerektirir; sunucu saati, kullanıcının yerel saati ve etkinlięin gerekleřtięi yerin saati farklı olabilir. Görelili zaman (“3 dakika önce”) yakın gemişte, mutlak zaman eski kayıtlarda daha iyi alışır; ikisi birlikte gerekiyorsa biri ipucu balonunda verilir.

Gerek zamanlı güncellemelerde (*real-time updates*) listenin üstüne yeni öęe eklenmesi, okumakta olan kullanıcının konumunu kaydırır. “3 yeni öęe” biçiminde bir düęmeyle güncellemeyi kullanıcının onayına bırakmak bu sorunu özer.

Erişilebilirlik

Erişilebilirlik, arayüzün engelli kullanıcılar tarafından da kullanılabilmesidir. Kapsayıcı tasarım (*inclusive design*) kavramı bu tanıma genişletir: kalıcı engellerin yanında geçici (kırık kol) ve duruma bağlı (güneş altında ekran, tek elle telefon kullanımı) kısıtları da kapsar. Bu genişletme pratik açıdan önemlidir, çünkü erişilebilirlik iyileştirmelerinin faydası küçük bir azınlıkla sınırlı değildir.

WCAG (*Web Content Accessibility Guidelines*), W3C tarafından yayımlanan erişilebilirlik standardıdır. A, AA ve AAA olmak üzere üç uyum düzeyi tanımlar; yasal düzenlemelerde ve pratik hedeflerde AA düzeyi esas alınır. Kriterler ölçülebilirdir: kontrast oranı, klavye erişimi, metin alternatifleri, odak görünürlüğü.

Anlamsal HTML (*semantic HTML*), her öge için işlevine uygun elementin kullanılmasıdır: düğme için `button`, başlık için `h2`, liste için `ul`. Bir `div` elementine tıklama olayı bağlamak görsel olarak aynı sonucu verir, ancak öge klavyeyle odaklanamaz, Enter ve Space tuşlarına yanıt vermez ve ekran okuyucu tarafından düğme olarak bildirilmez. Erişilebilirliğin sonradan eklenemeyeceği savının temeli budur; yanlış element seçimi yapısal bir karardır.

Klavye gezinimi (*keyboard navigation*), tüm işlevlerin fare olmadan yapılabilmesidir. Tab ve Shift+Tab ile ilerleme, Enter ve Space ile etkinleştirme, Escape ile kapatma, ok tuşlarıyla liste içinde hareket beklenen davranışlardır. Gezinme sırası DOM sırasını izlediği için, CSS ile görsel sırası değiştirilmiş öğelerde görsel sıra ile klavye sırası ayrışır; bu ayrışma gözle fark edilmez ancak klavye kullanıcısı için arayüzü kullanılamaz hale getirir.

Odak yönetimi (*focus management*), odağın nerede olduğunun denetlenmesidir. Kip pencere açıldığında odak pencerenin içine taşınmalı, kapandığında onu açan öğeye geri dönmelidir. Odak halkasının görünür olması şarttır; estetik gerekçeyle kaldırılması, klavye kullanıcısının nerede olduğunu göremez hale gelmesi demektir.

Odak kapanı (*focus trap*), kip pencere ya da menü açıkken odağın onun içinde kalmasıdır. Aksi halde Tab tuşu odağı arkadaki etkisiz sayfaya taşır ve kullanıcı görünmeyen öğeler arasında dolaşır.

Ekran okuyucu akışı, sayfanın sesli olarak sırayla okunmasıdır. Başlık hiyerarşisi atlanmamalı (`h2`'den `h4`'e geçilmemeli), görsellere metin alternatifi verilmeli, yalnızca ikon içeren düğmelere erişilebilir isim eklenmelidir.

ARIA (*Accessible Rich Internet Applications*), HTML'in kendi elementleriyle ifade edilemeyen anlamları eklemek için kullanılan öznitelik kümesidir: sekme, uyarı, canlı bölge gibi. Temel kural şudur: aynı işi yapan bir HTML elementi varsa ARIA kullanılmaz. Yanlış kullanılan ARIA, hiç ARIA kullanılmamasından daha zararlıdır, çünkü ekran okuyucuya yanlış bilgi verir.

Azaltılmış hareket (*reduced motion*), kullanıcının işletim sisteminde hareketi azalt tercihini belirtmesi durumunda animasyonların kapatılması ya da sadeleştirilmesidir. Bu tercih estetik değil tıbbidir; vestibüler bozukluğu olan kullanıcılarda büyük ölçekli hareket baş dönmesi ve bulantı üretir.

Yetki durumları da erişilebilirlik ve anlaşılabilirlik sorunudur. Kullanıcının yetkisi bulunmayan bir eylem tamamen gizlenebilir ya da etkisizleştirilip gerekçesi gösterilebilir. İkincisi genellikle daha iyidir, çünkü özelliğin varlığını öğretir ve

kullanıcı yetkiyi nereden isteyeceğini bilir.

Tasarım Sistemleri

Tasarım sistemi (*design system*), bileşenler, kurallar, değerler ve dokümantasyondan oluşan bütündür. Amacı, her yeni ekranda aynı kararların baştan verilmesini engellemektir. Yazılımdaki yeniden kullanılabilirlik ilkesinin arayüz karşılığıdır.

Tasarım değerleri (*design tokens*), renk, boşluk, köşe yarıçapı, gölge ve yazı ölçeği gibi değerlerin adlandırılmış halidir: **color-surface-primary**, **space-4**. Sabit sayıların doğrudan kullanılması yerine adlandırılmış değerlerin kullanılması, yazılımdaki “sihirli sayı kullanma” ilkesinin birebir karşılığıdır. Değer değiştiğinde onu kullanan her yer birlikte değişir.

Atomik tasarım (*atomic design*), Frost (2016) tarafından önerilen katmanlama yaklaşımıdır: atom (düğme, girdi alanı), molekül (arama kutusu), organizma (başlık çubuğu), şablon ve sayfa. Faydası, bir bileşen üzerinde çalışırken hangi soyutlama düzeyinde bulunduğunu netleştirmesidir.

Bileşen varyantları (*component variants*), aynı bileşenin tanımlı farklı halleridir: düğmenin birincil, ikincil, hayalet ve yıkıcı biçimleri; küçük, orta ve büyük boyutları. Yeni bir ihtiyaç doğduğunda yeni bir bileşen değil yeni bir varyant üretilir; aksi halde sistemde birbirine benzeyen ama farklı davranan çok sayıda bileşen birikir.

Stil kılavuzu (*style guide*), marka tarafındaki kuralları içerir: logo kullanımı, renkler, tipografi ve ton. Kalıp kütüphanesi (*pattern library*) ise bileşen değil senaryo çözümü toplar: silme onayının nasıl alınacağı, form hatasının nasıl gösterileceği.

Tasarım borcu (*design debt*), teknik borcun görsel karşılığıdır: tutarsız bileşenler, tek kullanımlık çözümler, sisteme hiç girmemiş kalıplar. Teknik borç gibi birikir ve tıpkı onun gibi ödenmediğinde her yeni ekranın maliyetini artırır.

Tasarım sisteminin sürümlemesi, bir bileşen değiştiğinde onu kullanan ekranların bozulmamasını gerektirir. Kırıcı değişikliklerin yönetimi burada da geçerlidir.

Arayüz değişikliklerinin yönetimi (*change management*) ayrı bir problemdir. Alışmış kullanıcı için değişiklik, iyileştirme olsa bile maliyettir. Önceden duyuru, geçiş dönemi ve mümkünse eski görünüme dönme seçeneği bu maliyeti düşürür. Kaldırılan bir özellik, kaldırılmadan önce uyarılmalı ve alternatifi gösterilmelidir.

Hareket ve Görsel Dil

Hareket

Yumuşatma (*easing*), hareketin hız eğrisidir. Doğrusal hareket yapay görünür, çünkü fiziksel dünyada hiçbir nesne sabit hızla başlayıp sabit hızla durmaz. Ekrana giren öge hızlı başlayıp yavaşlar, çıkan öge hızlanarak gider.

Süre, hareketin ne kadar sürdüğüdür. Küçük durum değişiklikleri 100–200 milisaniye, orta ölçekli geçişler 200–300, büyük ekran geçişleri 300–500 milisaniye aralığında tutulur. Uzun animasyon ilk kullanımda etkileyici, yüzüncü kullanımda engeldir; animasyon her gün defalarca görülecek bir şeydir.

Mekânsal süreklilik (*spatial continuity*), öğelerin nereden gelip nereye gittiğinin tutarlı olmasıdır. Sağdan açılan bir panel sağa doğru kapanır. Bu tutarlılık kullanıcının zihninde bir mekân kurar ve arayüzün öğrenilmesini kolaylaştırır.

Paylaşılan öge geçişi (*shared element transition*), bir öğenin iki ekran arasında sürekliliğini korumasıdır: listedeki kartın açılarak detay sayfasının başlığına dönüşmesi. İki ekran arasındaki bağı görsel olarak taşır.

Geri bildirim olarak hareket, animasyonun süs değil bilgi olmasıdır: silinen satırın kayarak kaybolması, kaydedilen öğenin kısa süre vurgulanması. Bilgi taşımayan animasyon yalnızca süreyi uzatır.

Görsel dil

İkon tasarımında tek stil kuralı geçerlidir: tüm ikonlar ya dolu ya çizgisel olmalı, aynı çizgi kalınlığını ve aynı köşe yaklaşımını paylaşmalıdır. İkonun tek başına anlam taşıdığı varsayımı çoğunlukla yanlıştır; yaygın birkaç ikon dışında metin etiketiyle birlikte kullanılmalıdır.

Yükseklik ve gölge (*elevation, shadow*), derinlik kurmanın aracıdır. Gölgenin şiddeti katman sırasını bildirir: kart hafif, açılır liste orta, kip pencere ağır gölge taşır. Gölge şiddeti rastgele değil, bir ölçeğe göre seçilir.

Köşe yarıçapı da bir ölçek değeridir. Küçük bileşenlerde küçük, büyük konteynerlerde büyük yarıçap kullanılır; karışık kullanım ürünü dağınık gösterir.

Görsel stil akımları arasındaki ayrım işlevsel bir sonuç doğurur. Beceriksiz gerçekçilik (*skeuomorphism*) fiziksel nesneleri taklit eder ve gösterenleri bol biçimde sağlar. Düz tasarım (*flat design*) bunları kaldırır ve tıklanabilirliği belirsizleştirir. Bugünkü yaygın yaklaşım, ölçülü derinlik kullanarak gösterenleri korumaktır.

Etik Boyut

Karanlık kalıplar (*dark patterns*), kullanıcıyı kendi çıkarına olmayan bir sonuca yönlendiren tasarımlardır; terim Brignull (2010) tarafından ortaya atılmıştır. Bu notta anlatılan kuralların tamamı, kullanıcının yararına olduğu kadar zararına da kullanılabilir; ayırım tekniğın kendisinde değil, kimin yararına uygulandığındadır.

Onaylatarak utandırma (*confirmshaming*), reddetme seçeneğinin kullanıcıyı küçük düşürecek biçimde yazılmasıdır: “Hayır, indirimlerden yararlanmak istemiyorum“. Reddetme eylemi, kabul eylemiyle aynı nötrlükte ifade edilmelidir.

Yem ve değiştirme (*bait and switch*), bir şey yapacağı izlenimi veren öğenin başka bir şey yapmasıdır. Kapatma ikonunun bir indirme başlatması tipik örnektir.

Hamamböceğı moteli (*roach motel*), girişı kolay çıkışı zor tasarlanmış akıştır: üyelik iki tıklamayla açılırken iptali telefon aramayı gerektirir. Bu kalıp çeşitli ülkelerde düzenlemeye konu olmuş ve kısmen yasaklanmıştır.

Onay arayüzleri (*consent UI*), çerez ve izin ekranlarıdır. Kabul etme ile reddetme seçeneklerinin eşit kolaylıkta sunulmaması bu kategoriye girer; “tümünü kabul et“ tek tıklamayken reddetmenin bir alt menüden geçmesi tasarlanmış bir sürtünmedir.

Bağımlılık üreten tasarım, sonsuz akış, kırmızı bildirim rozetleri ve çekerek yenileme gibi kalıpların kullanıcının dikkatini elde tutmak için kullanılmasıdır. Bu kalıpların etkili olduğu ölçülebilir; etik olup olmadığı ise ayrı bir sorudur ve ürün ekibinin kararıdır.

Tasarımda gizlilik (*privacy by design*), varsayılan ayarın en az veri toplayan seçenek olmasıdır. Varsayılanın güçlü etkisi göz önüne alındığında, veri toplamayı varsayılan yapmak teknik değil etik bir karardır.

Bilişsel yük (*cognitive load*), kullanıcıdan istenen zihinsel çabadır. Gereksiz seçim, hatırlama zorunluluğı ve kullanıcıya yaptırılan hesaplama bu yükü artırır. Nielsen’in kurallarından biri olan “hatırlamak yerine tanımak“, bu yükü azaltma ilkesidir: kullanıcıdan bir bilgiyi ezberlemesini istemek yerine, ihtiyaç duyduğu anda görünür kılmak.

Tasarım Süreci ve Ölçüm

Kullanıcı akışı (*user flow*), kullanıcının bir hedefe ulaşana kadar attığı adımların haritasıdır. Ekran tasarımından önce çıkarılır; akış çıkarılmadan tasarlanan ekranlar tek tek doğru, birlikte tutarsız olur.

Sadakat düzeyi (*fidelity*), bir tasarım çıktısının bitmişlik derecesidir: kaba çizim, tel çerçeve, yüksek sadakatli tasarım, etkileşimli prototip. Erken aşamada yüksek sadakatli çıktı üretmek tartışmayı yanlış yere çeker; yapı konuşulması gereken aşamada renk tartışılmaya başlanır.

Prototipleme, tıklanabilir bir akış üretmektir. Statik ekranların göremediği geçiş ve durum sorunlarını açığa çıkarır.

Kullanılabilirlik testi (*usability testing*), gerçek kullanıcının görevi yaparken izlenmesidir. Nielsen ve Landauer (1993), beş kullanıcının sorunların yaklaşık %85'ini ortaya çıkardığını göstermiştir; bu bulgu, testin pahalı ve büyük ölçekli olması gerekmediğini ortaya koyar. Testin temel kuralı sormak değil izlemektir: kullanıcı ne yaptığını değil, ne yapmak istediğini anlatır.

Sezgisel değerlendirme (*heuristic evaluation*), arayüzün bir kurallar listesine karşı denetlenmesidir; Nielsen'in on kuralı en yaygın kullanılan listedir. Kullanıcı bulmayı gerektirmediği için ucuzdur ve kullanılabilirlik testinden önce yapılır.

Görev başarı oranı ve görev süresi, kullanılabilirliğin en doğrudan iki ölçüsüdür: kullanıcı işi bitirebiliyor mu ve ne kadar sürede.

A/B testi, iki varyantın gerçek kullanıcılar üzerinde karşılaştırılmasıdır. Küçük, yerel değişikliklerde güvenilir sonuç verir; yön kararlarında yanıltıcıdır, çünkü yalnızca sunulan iki seçenek arasında karşılaştırma yapar ve üçüncü bir seçeneğin varlığını göstermez. Ayrıca kısa vadeli metrikleri iyileştiren değişiklikler uzun vadeli kullanıcı ilişkisini bozabilir.

Analitik veriler (ısı haritası, huni analizi, terk noktaları) sorunun nerede olduğunu gösterir, neden olduğunu göstermez. Nicel veri konumu, nitel araştırma nedeni verir; ikisi birbirinin yerine geçmez.

Tasarım eleştirisi (*design critique*), tasarımın yapılandırılmış biçimde değerlendirilmesidir. Beğenme üzerinden değil, belirlenmiş hedef üzerinden yürütülür.

Tasarım kalite denetimi (*design QA*), uygulanmış arayüzün tasarımla karşılaştırıldığı son adımdır. Boşluk, renk, durum ve hareket farkları bu aşamada yakalanır; bu adım atlandığında tasarım sistemi kâğıt üzerinde kalır.

Sonuç

Arayüz kurallarının iki farklı epistemolojik statüsü vardır ve bunları ayırmak pratik sonuç doğurur. Fitts, Hick ve Miller yasaları ile kontrast eşikleri ölçülmüş bilişsel ve fizyolojik sınırlardır; bunları “tercih meselesi“ sayan bir tartışma yanlış kurulmuştur. Buna karşılık bileşen seçimleri, düzen kalıpları ve görsel stil birikmiş uygulama gelenegidir; bağlama göre değişir ve tartışmaya açıktır. Bir ekip içindeki çoğu tasarım anlaşmazlığı, bu iki kümenin karıştırılmasından doğar.

Notta anlatılan kuralların önemli bir kısmı aynı ilkenin farklı katmanlardaki görünümüdür. Bilgi gizleme, kademeli açılım, bilişsel yükün azaltılması ve Hick yasası birbirinden bağımsız kurallar değil, tek bir ilkenin dört ayrı ölçekteki ifadesidir: kullanıcıya o an gerekmeyen şeyi gösterme. Benzer biçimde Gestalt prensipleri, ızgara sistemi ve tasarım değerleri, tutarlılığın algı, yerleşim ve kod katmanlarındaki karşılıklardır. Bu, listeyi ezberlemek yerine altındaki az sayıda ilkeyi kavramanın yeterli olduğunu gösterir.

Ön yüz geliştiricisi açısından çerçevenin en doğrudan sonucu şudur: bu kararların çoğu tasarımcı tarafından verilmez, çünkü statik tasarım çıktıları durumları, sınır koşullarını ve etkileşim ayrıntılarını içermez. Boş durumun ne diyeceği, hatanın nasıl gösterileceği, odağın kip pencere kapandığında nereye döneceği, çok uzun bir ismin nasıl kesileceği uygulama sırasında kararlaştırılır. Dolayısıyla bu bilgi geliştiricide bulunmadığında karar verilmemiş olmaz; yalnızca farkında olunmadan verilir.

Çerçevenin sınırları da belirtilmelidir. Kuralların çoğu Batı merkezli, soldan sağa okunan ve masüstü-mobil ekranlara dayalı bir bağlamda üretilmiştir; sağdan sola yazılan dillerde, farklı okuryazarlık düzeylerinde ve sesli ya da giyilebilir arayüzlerde geçerlilikleri yeniden sınanmalıdır. Ayrıca kuralların tamamı kullanıcının yararına yönelmiş değildir: aynı bilgi, dikkat ve karar üzerinde kullanıcı aleyhine bir güç de üretir. Bu nedenle arayüz bilgisi salt teknik bir yetkinlik olarak değil, uygulandığı yönün seçimini de içeren bir sorumluluk olarak taşınmalıdır.

Notun kapsamadığı alanlar şunlardır: sesli arayüzler, artırılmış ve sanal gerçeklik arayüzleri, veri görselleştirmenin kendi kural kümesi, oyun arayüzleri ve makine öğrenmesi tabanlı kişiselleştirmenin kullanıcı denetimiyle ilişkisi. Bunların her biri burada kurulan ilkeleri paylaşır ancak kendi sınırlarını ekler.

Kaynaklar

- Bringhurst, R. (1992). *The Elements of Typographic Style*. Hartley & Marks.
- Brignull, H. (2010). Dark Patterns. *darkpatterns.org* (sonradan *deceptive.design*).
- Doherty, W. J. & Thadani, A. J. (1982). *The Economic Value of Rapid Response Time*. IBM.

- Fitts, P. M. (1954). The information capacity of the human motor system in controlling the amplitude of movement. *Journal of Experimental Psychology*, 47(6).
- Frost, B. (2016). *Atomic Design*.
- Hick, W. E. (1952). On the rate of gain of information. *Quarterly Journal of Experimental Psychology*, 4(1).
- Hyman, R. (1953). Stimulus information as a determinant of reaction time. *Journal of Experimental Psychology*, 45(3).
- Kahneman, D., Fredrickson, B. L., Schreiber, C. A. & Redelmeier, D. A. (1993). When more pain is preferred to less. *Psychological Science*, 4(6).
- Krug, S. (2000). *Don't Make Me Think*. New Riders.
- Kurokawa, M. & Kashimura, K. (1995). Apparent usability vs. inherent usability. *CHI '95 Conference Companion*.
- Miller, G. A. (1956). The magical number seven, plus or minus two. *Psychological Review*, 63(2).
- Nielsen, J. (1994). *Usability Engineering*. Morgan Kaufmann. (On sezgisel kural.)
- Nielsen, J. (2000). Jakob's Law of Internet User Experience. *Nielsen Norman Group*.
- Nielsen, J. & Landauer, T. K. (1993). A mathematical model of the finding of usability problems. *INTERCHI '93*.
- Norman, D. A. (1988; gözden geçirilmiş baskı 2013). *The Design of Everyday Things*. Basic Books.
- Parnas, D. L. (1972). On the criteria to be used in decomposing systems into modules. *Communications of the ACM*, 15(12).
- Pirolli, P. & Card, S. (1999). Information foraging. *Psychological Review*, 106(4).
- von Restorff, H. (1933). Über die Wirkung von Bereichsbildungen im Spurenfeld. *Psychologische Forschung*, 18.
- W3C (2018). *Web Content Accessibility Guidelines (WCAG) 2.1*.
- Wertheimer, M. (1923). Untersuchungen zur Lehre von der Gestalt II. *Psychologische Forschung*, 4.
- Wroblewski, L. (2008). *Web Form Design*. Rosenfeld Media.

- Zeigarnik, B. (1927). Über das Behalten von erledigten und unerledigten Handlungen. *Psychologische Forschung*, 9.

Tesler yasası, Larry Tesler'a atfedilen ancak yazılı bir birincil kaynağı bulunmayan bir ilkedir; kökeni doğrulanmalıdır. 60-30-10 oranı iç mimari geleneginden gelen bir uygulama kuralıdır ve deneysel bir dayanağı yoktur.